Chapter 3 (Part 1)
Algorithms

MAD101

Ly Anh Duong

duongla3@fe.edu.vn

Table of Contents

Algorithms

1 Algorithms

Definition 1.1

An **algorithm** is a finite sequence of precise instructions (mã lệnh xác định) for performing a computation or for solving a problem.

Properties 1

1. **Input (Đầu vào).** An algorithm has input values from a specified set.
2. **Output (Đầu ra).** From each set of input values an algorithm produces output values from a specified set (solution).
3. **Definiteness (xác định).** The steps of an algorithm must be defined precisely.
4. **Correctness (chính xác).** An algorithm should produce the correct output values for each set of input values.
5. **Finiteness (hữu hạn).** An algorithm should produce the desired output after a finite (but perhaps large) number of steps for any input in the set.
6. **Effectiveness (hiệu quả).** It must be possible to perform each step of an algorithm exactly and in a finite amount of time.
7. **Generality (tổng quát).** The procedure should be applicable for all problems of the desired form.

ALGORITHM 1: Finding the Maximum Element

1 Algorithms

ALGORITHM 1 Finding the Maximum Element in a Finite Sequence.

procedure $max(a_1, a_2, \ldots, a_n$: integers)
$max := a_1$
for $i := 2$ **to** n
    if $max < a_i$ **then** $max := a_i$
return max {max is the largest element}

Example 1.1.
Finding the Maximum Element in a Finite Sequence $\{8, 4, 11, 3, 10\}$.

Solution of Example 1.1

procedure max $(a_1, a_2, a_3, a_4, a_5$: integers)
max:=a_1
for i:=2 **to** 5
if max $< a_i$ **then** max:= a_i
return max {max is the largest element}

max=8
$i = 2(8 \geq 4) \rightarrow$ max=8
$i = 3(8 < 11) \rightarrow$ max=11
$i = 4(11 \geq 3) \rightarrow$ max=11
$i = 5(11 \geq 10) \rightarrow$ max=11

Thus, max=11

ALGORITHM 2: The Linear Search

1 Algorithms

ALGORITHM 2 The Linear Search Algorithm.

procedure *linear search*(x: integer, $a_1, a_2, \ldots, a_n$: distinct integers)
$i := 1$
while ($i \leq n$ and $x \neq a_i$)
    $i := i + 1$
if $i \leq n$ **then** *location* $:= i$
else *location* $:= 0$
return *location*{*location* is the subscript of the term that equals x, or is 0 if x is not found}

Example 1.2. List all the steps used to search for 9 in the sequence $2, 3, 4, 5, 6, 8, 9, 11$ using a **linear search**. How many comparisons required to search for 9 in the sequence.

Solution of Example 1.2

procedure linear search (x: integer, $a_1, a_2, ..., a_8$: distinct integers)

$i := 1$

while ($i \leq 8$ and $x \neq a_i$)

$i := i + 1$

if $i \leq 8$ **then** location: $= i$

else location: $= 0$

return location {location is the subscript of the term that equals x, or is 0 if x is not found}

i = 1
(1 ≤ 8 and 9 ≠ 2) ➔ i:=i+1 = 2
i = 2
(2 ≤ 8 and 9 ≠ 3) ➔ i:= i+1 = 3
i = 3
(3 ≤ 8 and 9 ≠ 4) ➔ i:= i+1 = 4
i = 4
(4 ≤ 8 and 9 ≠ 5) ➔ i:= i+1 = 5
i = 5
(5 ≤ 8 and 9 ≠ 6) ➔ i:= i+1 = 6
i = 6
(6 ≤ 8 and 9 ≠ 8) ➔ i:= i+1 = 7
i = 7
(7 ≤ 8 and 9 ≠ 9) // the condition is false
7 ≤ 9 ➔ location = 7.

Based on the steps above, there are 15 comparisons (≤, ≠) required.

ALGORITHM 3: The Binary Search

ALGORITHM 3 The Binary Search Algorithm.

procedure *binary search* (x: integer, $a_1, a_2, \ldots, a_n$: increasing integers)
$i := 1$ {i is left endpoint of search interval}
$j := n$ {j is right endpoint of search interval}
while $i < j$
    $m := \lfloor (i + j)/2 \rfloor$
    if $x > a_m$ **then** $i := m + 1$
    else $j := m$
if $x = a_i$ **then** *location* $:= i$
else *location* $:= 0$
return *location* {*location* is the subscript i of the term a_i equal to x, or 0 if x is not found}

Example 1.3. To search for 19 in the list

$$1, 2, 3, 5, 6, 7, 8, 10, 12, 13, 15, 16, 18, 19, 20, 22$$

Solution of Example 1.3

procedure binary search (x: integer, $a_1, a_2, \ldots, a_{16}$: increasing integers)

$i := 1$ { i is left endpoint of search interval}

$j := 16$ { j is right endpoint of search interval}

while $i < j$

$m := \lfloor (i + j)/2 \rfloor$

if $x > a_m$ **then** $i := m + 1$

else $j := m$

if $x = a_i$ **then** location: $= i$

else location: $= 0$

return location {location is the subscript of the term that equals x, or is 0 if x is not found}

Solution of Example 1.3

$i = 1,\ j = 16$

while $i < j$

$m = \lfloor \frac{1+16}{2} \rfloor = \lfloor 8.5 \rfloor = 8$

$19 > 10 \rightarrow i = 8 + 1 = 9,\ 19 \neq 12 \rightarrow$location:=0

$m = \lfloor \frac{9+16}{2} \rfloor = \lfloor 12.5 \rfloor = 12$

$19 > 16 \rightarrow i = 12 + 1 = 13,\ 19 \neq 18 \rightarrow$location:=0

$m = \lfloor \frac{13+16}{2} \rfloor = \lfloor 14.5 \rfloor = 14$

$19 \leq 19 \rightarrow j = 14,\ 19 \neq 18 \rightarrow$location:=0

$m = \lfloor \frac{13+14}{2} \rfloor = \lfloor 13.5 \rfloor = 13$

$19 > 18 \rightarrow i = 13 + 1 = 14$

$19 = 19$

$\longrightarrow$ location $= 14$

Sorting

1 Algorithms

Definition 1.2

Sorting is putting the elements into a list in which the elements are in increasing order.
For instance, sorting the list 7, 2, 1, 4, 5, 9 produces the list 1, 2, 4, 5, 7, 9. Sorting the list d, h, c, a, f (using alphabetical order) produces the list a, c, d, f, h.

There are some sorting algorithms:

1. Bubble sort
2. Insertion sort
3. Selection sort (exercise p.178)
4. Binary insertion sort (exercise p.179)
5. Shaker sort (exercise p.259)
6. Merge sort and quick sort (section 4.4)
7. Tournament sort (section 10.2)

ALGORITHM 4: The Bubble Sort

1 Algorithms

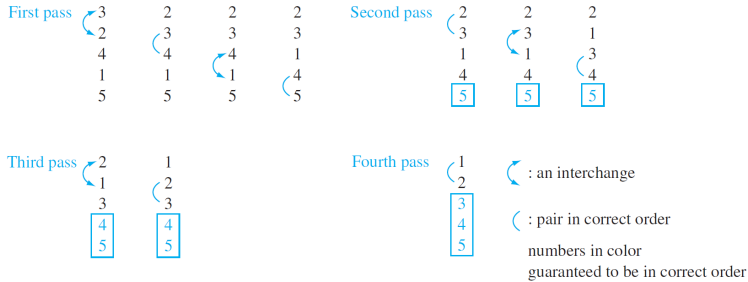
ALGORITHM 4 The Bubble Sort.

procedure *bubblesort*($a_1, \ldots, a_n$: real numbers with $n \geq 2$)
for $i := 1$ **to** $n - 1$
    for $j := 1$ **to** $n - i$
        if $a_j > a_{j+1}$ **then** interchange a_j and a_{j+1}
{$a_1, \ldots, a_n$ is in increasing order}

Example 1.4. Use the bubble sort to put $3, 2, 4, 1, 5$ into increasing order.

Solution of example 1.4

1 Algorithms



procedure bubble sort ($a_1, a_2, \ldots, a_5$:real numbers with $5 \geq 2$)
for $i := 1$ **to** 4
for $j := 1$ **to** $5 - i$
if $a_j > a_{j+1}$ **then** interchange a_j and a_{j+1}
$\{a_1, a_2, \ldots, a_5$ is in increasing order$\}$

Solution of example 1.4

$i = 1$

$j = 1 : 3 > 2 \rightarrow 2, 3, 4, 1, 5$

$j = 2 : 3 \leq 4 \rightarrow 2, 3, 4, 1, 5$

$j = 3 : 4 > 1 \rightarrow 2, 3, 1, 4, 5$

$j = 4 : 4 \leq 5 \rightarrow 2, 3, 1, 4, 5$

$i = 2$

$j = 1 : 2 \leq 3 \rightarrow 2, 3, 1, 4, 5$

$j = 2 : 3 > 1 \rightarrow 2, 1, 3, 4, 5$

$j = 3 : 3 \leq 4 \rightarrow 2, 1, 3, 4, 5$

$i = 3$

$j = 1 : 2 > 1 \rightarrow 1, 2, 3, 4, 5$

$j = 2 : 2 \leq 3 \rightarrow 1, 2, 3, 4, 5$

$i = 4$

$j = 1 : 1 \leq 2 \rightarrow 1, 2, 3, 4, 5$

ALGORITHM 5: The Insertion Sort

1 Algorithms

ALGORITHM 5 The Insertion Sort.

procedure *insertion sort*($a_1, a_2, \ldots, a_n$: real numbers with $n \geq 2$)
for $j := 2$ **to** n
    $i := 1$
    while $a_j > a_i$
        $i := i + 1$
    $m := a_j$
    for $k := 0$ **to** $j - i - 1$
        $a_{j-k} := a_{j-k-1}$
    $a_i := m$
{$a_1, \ldots, a_n$ is in increasing order}

Example 1.5. Use the insertion sort to put the elements of the list 3, 2, 4, 1, 5 in increasing order.

Solution of example 1.5

1 Algorithms

procedure insertion sort ($a_1, a_2, \ldots, a_5$:real numbers with $5 \geq 2$)

for $j := 2$ **to** 5 *{j: position of the examined element}*
{finding out the right position of a_j*}*

$i := 1$
while $a_j > a_i$
$i := i + 1$
$m := a_j$ *{ save* a_j *}*
{moving j-i elements backward}
for $k := 0$ **to** $j - i - 1$
$a_{j-k} := a_{j-k-1}$
{move a_j *to the position i}*
$a_i := m$

$\{a_1, a_2, \ldots, a_5$ is increasing order$\}$

Solution of example 1.5

$j = 2$

$i = 1, 2 \leq 3 \rightarrow i = 1, m = 2$

$k = 0 \rightarrow a_2 = 3, a_1 = 2$

$\rightarrow 2, 3, 4, 1, 5$

$j = 3$

$i = 1, 4 > 2$

$i = 2, 4 > 3$

$\rightarrow 2, 3, 4, 1, 5$

$j = 4$

$i = 1, 1 \leq 2 \rightarrow i = 1, m = 1$

$k = 0 \rightarrow a_4 = 4$

$k = 1 \rightarrow a_3 = 3$

$k = 2 \rightarrow a_2 = 2$

$a_1 = 1$

$\rightarrow 1, 2, 3, 4, 5$

$j = 4$

$i = 1, 1 \leq 2 \rightarrow i = 1, m = 1$

$k = 0 \rightarrow a_4 = 4$

$k = 1 \rightarrow a_3 = 3$

$k = 2 \rightarrow a_2 = 2$

$a_1 = 1$

$\rightarrow 1, 2, 3, 4, 5$

$j = 5$

$i = 1, 5 > 1$

$i = 2, 5 > 2$

$i = 3, 5 > 3$

$i = 4, 5 > 4$

Thus, $\rightarrow 1, 2, 3, 4, 5$

Greedy Algorithm

- They are usually used to solve optimization problems: Finding out a solution to the given problem that either minimizes or maximizes the value of some parameter.
- Selecting the best choice at each step, instead of considering all sequences of steps that may lead to an optimal solution.
- Some problems:
  1. Finding a route between two cities with smallest total mileage (number of miles that a person passed).
  2. Determining a way to encode messages using the fewest bits possible.
  3. Finding a set of fiber links between network nodes using the least amount of fiber.

Table of Contents

Introduction

2 The Growth of Functions

1. The complexity of an algorithm that acts on a sequence depends on the number of elements of sequence.
2. The growth of a function is an approach that help selecting the right algorithm to solve a problem among some of them.
3. Big-Oh notation is a mathematical representation of the growth of a function.

Example 2.1.

Algorithm	Big O Complexity
Binary Search	$O(\log n)$
Bubble Sort	$O(n^2)$
Linear Search	$O(n)$
Dijkstra	$O(n^2)$

Big-O Notation

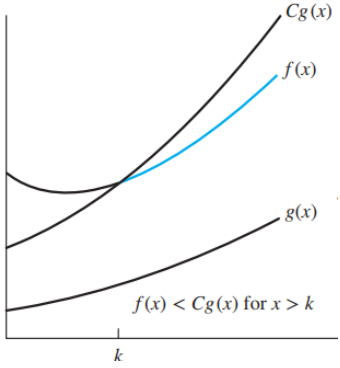
2 The Growth of Functions

Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that $f(x)$ is $O(g(x))$ if there are constants C and k such that

$$|f(x)| \leq C|g(x)|$$

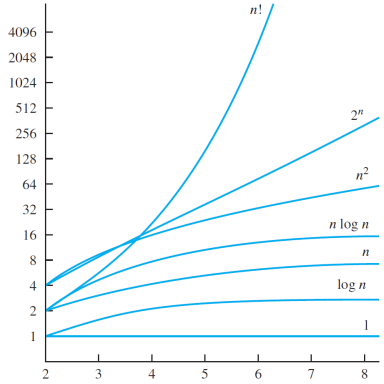
whenever $x > k$.
[Read: "$f(x)$ is big-oh of $g(x)$"].

Example 2.2.
Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$.



The Growth of Combinations of Functions

2 The Growth of Functions



$$1 < logn < n < nlogn < n^2 < 2^n < n!$$

Note 2.1. $Cf(n) \sim f(n)$, C is constant; $logn \leq n^{\alpha}$, $\forall \alpha > 0$.

Big-O: Theorem

2 The Growth of Functions

Theorem 2.1

Let $f(x) = a_n x^n + a_{n1} x^{n1} + \cdots + a_1 x + a_0$, where $a_0, a_1, \ldots, a_{n1}, a_n$ are real numbers. Then $f(x)$ is $O(x_n)$.

Theorem 2.2

1. $f_1(x) = O(g_1(x)) \wedge f_2(x) = O(g_2(x))$

$$\Longrightarrow (f_1 + f_2)(x) = O(max(|g_1(x)|, |g_2(x)|))$$

2. $f_1(x) = O(g_1(x)) \wedge f_2(x) = O(g_2(x))$

$$\Longrightarrow (f_1 f_2)(x) = O(g_1 g_2(x))$$

Example 2.3.

1. Estimate big-oh of functions $100n^2 + 23nlogn + 2019$
2. Give a big-O estimate for $(2n^2 + 17n)(7logn + 15)$

Quiz

2 The Growth of Functions

Q1. The function
$f(x) = 4^x + x^5 + 2\log x$ is ...
Select one:

a. $O(x^5)$
b. $O(2^x)$
c. $O(5^x)$
d. $O(x^4)$

Q2. Which of the following functions is big-oh of n?
Select one or more:

a. $g(n) = \log n + 2018$
b. $g(n) = 2018n + 1$
c. $g(n) = n\log n + 7$
d. $g(n) = n^2 - 2n$

Big-Omega and Big-Theta Notation

Big-Omega: $\exists C > 0,\ k :\ x \geq k \wedge |f(x)| \geq C|g(x)| \implies f(x) = \Omega(g(x))$.

Big-Theta: $\begin{cases} f(x) = O(g(x)) \\ f(x) = \Omega(g(x)) \end{cases} \implies f(x) = \Theta(g(x))$

Example 2.4. Show that $f(x) = 1 + 2 + ... + n$ is $\Theta(n^2)$.

Big-Omega and Big-Theta Notation

2 The Growth of Functions

Big-Omega: $\exists C > 0,\ k :\ x \geq k \wedge |f(x)| \geq C|g(x)| \Longrightarrow f(x) = \Omega(g(x))$.

Big-Theta: $\begin{cases} f(x) = O(g(x)) \\ f(x) = \Omega(g(x)) \end{cases} \Longrightarrow f(x) = \Theta(g(x))$

Example 2.5. Show that $f(x) = 1 + 2 + ... + n$ is $\Theta(n^2)$.

Solution.

$$f(x) = 1 + 2 + ... + n = \frac{n(n+1)}{2} \Longrightarrow \frac{n^2}{2} \leq f(x) \leq n^2 \Longrightarrow f(x) = \Theta(n^2)$$

Note 2.3.

1. $f(x)$ is $\Omega(g(x))$ **if and only if** $g(x)$ is $O(f(x))$.
2. If $f(x) = \Theta(g(x))$ then f(x) is of order g(x) or f(x) and g(x) are of the same order (cùng độ tăng).
3. Let $f(x) = a_n x^n + a_{n-1}x^{n-1} + ... + a_1 x + a_0$ where $a_1, a_2, ...a_{n-1}, a_n$ are real numbers. Then, $f(x)$ is $\Theta(x^n)$.

Table of Contents

- Computational complexity = **Time complexity** + Space complexity.
- Time complexity can be expressed in terms of the number of operations used by the algorithm.
  1. **Worst-case** complexity (tells us how many operations an algorithm requires to guarantee that it will produce a solution.).
  2. **Average-case** complexity (the average number of operations used to solve the problem over all possible inputs of a given size).

Demo 1

3 Complexity of Algorithms

Describe the time complexity (Worst-case) of the ALGORITHM 1

ALGORITHM 1 Finding the Maximum Element in a Finite Sequence.

procedure $max(a_1, a_2, \ldots, a_n$: integers)
$max := a_1$
for $i := 2$ **to** n
    if $max < a_i$ **then** $max := a_i$
return max {max is the largest element}

i	Number of compairisons
2	2
3	2
...	2
n	2
n+1	1, $max < a_i$ is omitted

$\Longrightarrow$ Number of comparisons
$f(n) = 2(n-1) + 1 = 2n - 1.$
Thus, $f(x) = \Theta(n)$.

Example

Example 3.1.

```
procedure printsth(n: positive integer)
    for i:=1 to n do
        print "hi"
    for k:=1 to n do
        print "I love you"
```

Estimate big-O of the given algorithm

Solution. f(n)=n+n=2n is O(n)

Example 3.2.

```
procedure printsth(n: positive integer)
    for i:=1 to n do
        for k:=1 to n do
            print "I love you"
```

Estimate big-O of the given algorithm

Solution. $f(n) = n.n = n^2$ is $O(n^2)$

Understanding the Complexity of Algorithms

- The linear search algorithm has linear (worst-case or average-case) complexity.
- The binary search algorithm has logarithmic (worst-case) complexity.
- Many important algorithms have $n \log n$, or linearithmic (worst-case) complexity, such as the merge sort (Chapter 4).

TABLE 1 Commonly Used Terminology for the Complexity of Algorithms.

Complexity	*Terminology*
$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \log n)$	Linearithmic complexity
$\Theta(n^b)$	Polynomial complexity
$\Theta(b^n)$, where $b > 1$	Exponential complexity
$\Theta(n!)$	Factorial complexity

Note 3.1.

Linearithmic = Linear + Logarithmic.

Q&A

Thank you for listening!